JADAVPUR UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assignment 3 Report

Design and Implementation of Medium Access Control Mechanisms

Empirical Evaluation of Non-Persistent, 1-Persistent, p-Persistent, and CSMA/CD Protocols over a Simulated Socket Shared Channel

Course: Computer Networks Laboratory (BCSE-III)

Student Name: Mandeep Shaw

Section: A1

Roll Number: 302510501002

Assignment: Assignment 3 (Medium Access Control: CSMA Techniques with Collision Detection)

Date of Submission: November 2026

Contents

1 Introduction and Objectives **1**

2 Theoretical Principles of Medium Access Control **1**
2.1 Channel Propagation Delay and Normalized Parameter a . . . 1
2.2 Non-Persistent CSMA . . . 2
2.3 1-Persistent CSMA . . . 2
2.4 p-Persistent CSMA and Slotted Arbitration . . . 2
2.5 CSMA with Collision Detection (CSMA/CD) . . . 3

3 System Architecture and Procedural Design **3**
3.1 3-Tier Client-Server Architecture Overview . . . 3
3.2 Frame Structure and In-Memory Layout . . . 4
3.3 Channel State Machine (FSM) . . . 4
3.4 Protocol Opcodes and Channel Handshake . . . 5

4 Implementation Details **5**
4.1 Key Methods and Interfaces . . . 5
4.2 Core Code Snippets . . . 5

5 Test Cases and Experimental Methodology **7**

6 Experimental Results and Empirical Evaluation **7**
6.1 Test Case 1: p-Persistent Performance Dynamics (Fixed $N = 10$) . . . 7
6.2 Test Case 2: Comparative Evaluation across Contending Stations N . . . 8

7 Live Execution Traces and Socket Transcripts **8**
7.1 Trace 1: CSMA/CD Collision Detection, Jam Signal, and Immediate Abort . . . 8
7.2 Trace 2: 1-Persistent CSMA Post-Busy Synchronized Collision Cascade . . . 9
7.3 Trace 3: Non-Persistent CSMA Carrier Sensing and Random Deferral . . . 9
7.4 Trace 4: p-Persistent CSMA Slotted Deferral and Acquisition . . . 9

8 Analysis and Discussion **10**
8.1 Impact of Persistence Probability p in p-Persistent CSMA (Test Case 1) . . . 10
8.2 Comparative Protocol Efficiency across Contending Stations N (Test Case 2) . . . 10
8.3 Known Limitations and Practical Enhancements . . . 10

9 Comments and Learning Evaluation **10**
9.1 Evaluation of the Lab Assignment . . . 10
9.2 What I Learned . . . 11
9.3 Difficulty Assessment . . . 11
9.4 Suggested Improvements . . . 11

10 Sources Used **11**

1 Introduction and Objectives

In broadcast and shared-medium communications, multiple independent network nodes share a common physical channel to transmit data packets. Without centralized scheduling or coordinated arbitration, concurrent transmissions on the shared medium inevitably overlap in time and space, resulting in destructive interference known as a **packet collision**. When a collision occurs, colliding frames are rendered mutually unreadable, corrupted, and discarded by receivers, leading to substantial degradation of throughput, increased latency, and wasted bandwidth.

To resolve contention and allocate channel access fairly and efficiently among autonomous stations, the Data Link Layer—specifically the **Medium Access Control (MAC) sublayer**—employs Carrier Sense Multiple Access (CSMA) techniques. CSMA mandates that a station sense the medium (carrier sensing) prior to transmitting, adopting the foundational principle "listen before talk."

The primary objectives of this laboratory assignment are:

1. **Emulation of a Shared Physical Channel Server:** Design and construct a multi-client socket-based shared channel emulator in C running over TCP/IP loopback (`127.0.0.1:5000`) that dynamically tracks and enforces the physical channel state machine among three distinct states: **IDLE**, **BUSY**, and **COLLISION**.
2. **Implementation of Independent Station Processes:** Construct independent client processes emulating network stations implementing four canonical medium access strategies:
   - **Non-Persistent CSMA:** Randomizes re-sensing intervals upon detecting a busy medium, reducing post-transmission collision bursts at the cost of idle channel slots.
   - **1-Persistent CSMA:** Continuously monitors the medium and transmits with probability $p = 1$ immediately when the medium becomes idle, maximizing channel utilization under low traffic but suffering severe collision cascades under heavy loads.
   - **p-Persistent CSMA:** Balances the trade-off between idle channel waste and collision bursts by operating on slotted contention boundaries, transmitting with probability p and deferring with probability $1 - p$.
   - **CSMA with Collision Detection (CSMA/CD):** Emulates IEEE 802.3 Ethernet behavior where transceivers monitor the medium while transmitting, immediately abort collided transmissions, emit a jamming signal, and schedule retransmissions using Truncated Binary Exponential Backoff (BEB).
3. **End-to-End Three-Tier Architecture:** Implement an end-to-end communication pipeline: Senders $\rightarrow$ Channel Server $\rightarrow$ Receiver, where clean frames are relayed to a dedicated destination node and acknowledgments are routed back for collision-detected transfers.
4. **Empirical Performance Profiling:** Benchmark the protocols across the mandatory test cases:
   - Sweep persistence probability $p \in [0.05, 1.00]$ in p-Persistent CSMA for fixed station count ($N = 10$) to determine the optimal operating point p^*.
   - Benchmark all four schemes across varying numbers of contending stations ($N \in [2, 30]$) to analyze collisions, average transmission delay, and channel throughput.

2 Theoretical Principles of Medium Access Control

2.1 Channel Propagation Delay and Normalized Parameter a

In any broadcast medium, electromagnetic signals propagate at a finite velocity v (approximately 2×10^8 m/s in copper cabling). For two stations separated by distance d, the one-way propagation delay is:

$$\tau = T_{\text{prop}} = \frac{d}{v} \tag{1}$$

The vulnerability period of CSMA is 2τ: if Station A begins transmitting at time t_0, its signal does not reach Station B until $t_0 + \tau$. If Station B senses the channel at $t_0 + \tau - \epsilon$, it detects an IDLE medium and begins transmitting, causing a collision that corrupts both packets. The normalized propagation

delay parameter a is defined as:

$$a = \frac{T_{\text{prop}}}{T_{\text{tx}}} = \frac{\tau \cdot B}{L_{\text{frame}}} \tag{2}$$

where B is the channel bandwidth and L_{frame} is the frame length in bits. As $a \to 0$, carrier sensing becomes instantaneous and channel efficiency approaches theoretical capacity. When $a \gg 1$, carrier sensing fails because stations sense outdated channel states.

2.2 Non-Persistent CSMA

In Non-Persistent CSMA, a station with a frame ready for transmission acts as follows:

1. Senses the shared channel.
2. If the channel is IDLE, the station transmits immediately.
3. If the channel is BUSY, the station does not continuously monitor the channel. Instead, it waits for an independent random backoff interval drawn from a random distribution and then re-senses the channel.

Because contending stations that sense a busy channel back off for different random durations, Non-Persistent CSMA substantially dampens the probability that multiple stations transmit simultaneously the moment a current transmission terminates. However, this randomness introduces idle slots when the channel is unoccupied, causing lower channel utilization under light loads. The theoretical throughput S as a function of offered channel traffic G (in frames per transmission time) is given by:

$$S_{\text{Non-Pers}} = \frac{G \cdot e^{-aG}}{G(1+2a) + e^{-aG}} \tag{3}$$

2.3 1-Persistent CSMA

In 1-Persistent CSMA, stations operate with maximum greediness:

1. Senses the shared channel.
2. If the channel is IDLE, transmits immediately with probability $p = 1$.
3. If the channel is BUSY, persists sensing the channel continuously until it detects the transition to IDLE, and then transmits immediately.

Under light traffic, 1-Persistent CSMA incurs zero unnecessary idle waiting time. However, if two or more stations generate packets while another transmission is active, all waiting stations sense the channel becoming idle at the exact same instant and transmit simultaneously. This produces a **deterministic post-transmission collision cascade**. Theoretical throughput for 1-Persistent CSMA is formulated as:

$$S_{\text{1-Pers}} = \frac{G[1 + G + aG(1 + G + \frac{aG}{2})]e^{-G(1+2a)}}{G(1+2a) - (1 - e^{-aG}) + (1 + aG)e^{-G(1+a)}} \tag{4}$$

Under the assumption of negligible propagation delay ($a \to 0$), this simplifies to:

$$S_{\text{1-Pers}} = \frac{G(1+G)e^{-G}}{G + e^{-G}} \tag{5}$$

Under heavy traffic ($G \gg 1$), $S_{\text{1-Pers}} \to 0$ due to catastrophic collision cascades.

2.4 p-Persistent CSMA and Slotted Arbitration

The p-Persistent CSMA scheme combines the prompt responsiveness of 1-Persistent CSMA with the collision avoidance of Non-Persistent CSMA by operating on slotted contention boundaries (where slot length $\delta \geq \tau$):

1. When a station has a packet and senses the channel idle, it transmits with probability p, or defers to the next slot with probability $q = 1 - p$.
2. If it defers to the next slot, it senses the channel again:
   - If the slot is still IDLE, it transmits with probability p, or defers with probability q.

- If the channel has become BUSY (another station began transmitting), it acts as though a collision occurred and schedules retransmission according to backoff.

If N stations are ready to transmit when the channel clears, the probability that exactly one station successfully acquires the medium in the first slot is:

$$P_{\text{success}} = N \cdot p \cdot (1-p)^{N-1} \tag{6}$$

The optimal probability p^* that maximizes P_{success} is derived by differentiating with respect to p:

$$\frac{dP_{\text{success}}}{dp} = N(1-p)^{N-1} - N(N-1)p(1-p)^{N-2} = 0 \implies p^* = \frac{1}{N} \tag{7}$$

Thus, under $N = 10$ contending stations, the optimal persistence probability lies in the region $p^* \approx 0.10$. Selecting $p \ll p^*$ wastes channel capacity on idle slots, while $p \gg p^*$ causes severe collision bursts.

2.5 CSMA with Collision Detection (CSMA/CD)

Pure CSMA protocols suffer from a major inefficiency: once two stations collide, they continue transmitting their complete frames to the end, wasting an entire transmission duration T_{tx}. In IEEE 802.3 Ethernet, stations implement **Collision Detection (CSMA/CD)**:

1. **Listen While Talking:** While transmitting, the transceiver continuously compares the transmitted signal with the channel state. If simultaneous transmission is detected, a collision is flagged.
2. **Immediate Abort and Jam Signal:** Upon detecting a collision, the station halts transmission immediately and transmits a 32-to-48 bit **Jamming Signal** to ensure all contending stations detect the collision.
3. **Truncated Binary Exponential Backoff (BEB):** Colliding stations defer their next attempt by R slot times, where R is drawn uniformly at random:

$$R \in [0, 2^k - 1], \quad \text{where } k = \min(\text{attempt}, 10) \tag{8}$$

   If collisions persist after 16 attempts (MAX_COLLISIONS = 16), the frame is dropped with an unrecoverable medium error.

To ensure that a station does not complete transmitting before the collision signal returns from the furthest point of the collision domain, the frame transmission time must exceed the round-trip propagation time:

$$T_{\text{tx}} \geq 2 \cdot \tau \implies L_{\min} \geq 2 \cdot \tau \cdot B = 2 \cdot d_{\max} \cdot \frac{B}{v} \tag{9}$$

This fundamental relationship dictates the 64-byte minimum frame size in standard IEEE 802.3 Ethernet.

3 System Architecture and Procedural Design

3.1 3-Tier Client-Server Architecture Overview

The medium access emulation environment is structured as a modular 3-tier client-server socket pipeline operating over TCP/IP loopback (`127.0.0.1:5000`). The architecture comprises three distinct entities:

- **Shared Channel Server (`server.c`):** Acts as the physical transmission medium. It manages the global channel state machine (`CH_IDLE`, `CH_BUSY`, `CH_COLLISION`), multiplexes incoming carrier sense requests, detects simultaneous transmission overlaps, broadcasts jamming signals (in CSMA/CD) or drops collided frames naturally (in pure CSMA), and forwards clean frames to the destination receiver.
- **Contending Network Stations (`sender.c`):** Independent processes representing network interface cards (NICs). Each station reads from `input.txt`, segments data into 46-byte payloads, builds 65-byte Ethernet frames, executes carrier sensing, implements backoff, and tracks per-frame delay and collisions.

- **Destination Receiver Node (`receiver.c`):** Registers with the channel server, receives clean forwarded frames, unpacks headers, displays frame statistics, and issues 2-byte ACKs (`0xA5, seqNo`) for CSMA/CD transmissions.

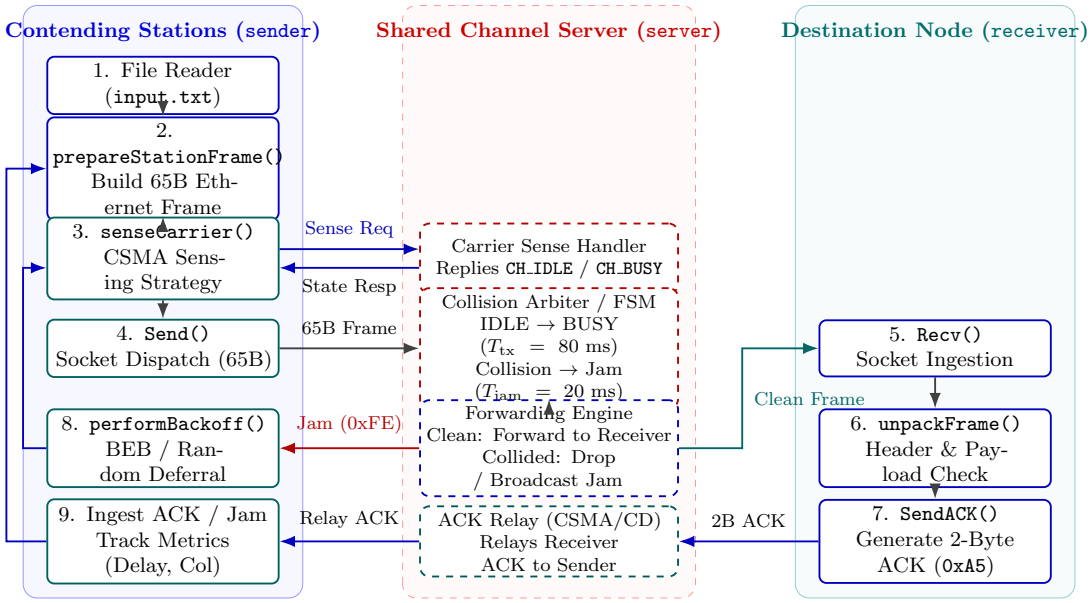


Figure 1: Three-tier procedural organization of the multi-station CSMA shared channel simulation over loopback.

3.2 Frame Structure and In-Memory Layout

Conforming strictly to the preceding lab assignments, every transmission uses a fixed 65-byte Ethernet-style frame consisting of a 15-byte header, a 46-byte payload, and a 4-byte Frame Check Sequence (FCS) trailer.

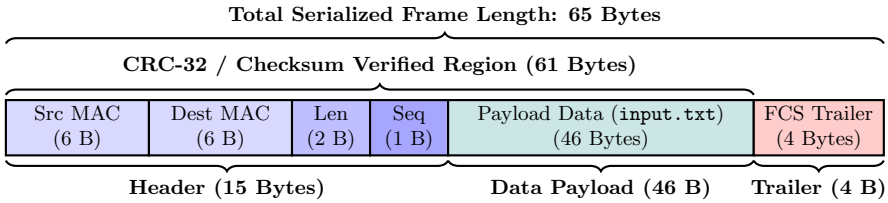


Figure 2: In-memory layout and serialization structure of the 65-byte Data Link Frame.

The acknowledgment frame is compact (2 bytes):

$$\text{ACK Frame} = [\text{Marker (0xA5) (1 Byte)} \mid \text{Ack Sequence Number (1 Byte)}] \tag{10}$$

3.3 Channel State Machine (FSM)

The shared channel server maintains an explicit finite state machine with three states:

- **IDLE (0):** No station is transmitting. Carrier sensing replies with `CH_IDLE`.
- **BUSY (1):** Exactly one station is transmitting. The channel remains occupied for $T_{\text{tx}} = 80$ ms. If another station transmits during this interval, a collision is flagged.
- **COLLISION (2):** Two or more transmissions overlap. For pure CSMA, the medium remains occupied for the remaining transmission duration and collided frames are naturally dropped. For CSMA/CD, the server broadcasts a 2-byte Jam Signal (`0xFE, 0x02`) to colliding stations and clears the medium after $T_{\text{jam}} = 20$ ms.

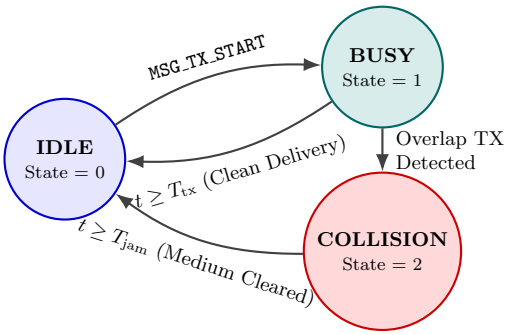


Figure 3: Finite state machine of the shared physical transmission medium.

3.4 Protocol Opcodes and Channel Handshake

The entities coordinate through standardized control opcodes defined in `csma_common.h`:

- `MSG_REGISTER_RX` (`0x01`): Receiver registers with the server as the destination sink.
- `MSG_REGISTER` (`0x05`): Sender registers its station name string.
- `MSG_PROTO_SYNC` (`0x06`) / `MSG_SET_PROTO` (`0x07`): The first connecting station selects the CSMA protocol; the server locks this selection and enforces it across all stations.
- `MSG_SENSE_REQ` (`0x10`) / `MSG_SENSE_RESP` (`0x11`): Sender senses carrier; server replies with `CH_IDLE` or `CH_BUSY`.
- `MSG_TX_START` (`0x20`): Dispatches a 65-byte packed Ethernet frame onto the medium.
- `MSG_FWD_FRAME_NOACK` (`0x30`) / `MSG_FWD_FRAME_ACK` (`0x31`): Server forwards frame to receiver.
- `MSG_TX_ACK` (`0xA5`): 2-byte ACK (`0xA5, seqNo`) relayed to the transmitting station.
- `MSG_COLLISION` (`0xFE`): Jam signal broadcast to colliding stations in CSMA/CD.
- `MSG_STATION_DONE` (`0xFF`): Sender signals completion of all frames.

4 Implementation Details

4.1 Key Methods and Interfaces

Table 1 maps the programmatic methods implemented across the three software modules.

Entity	Method Name	Functional Description
`sender.c`	`senseCarrier()`	Sends `MSG_SENSE_REQ` over socket and reads 2-byte state response.
	`performBackoff()`	Executes random backoff (Non-Persistent) or Truncated Binary Exponential Backoff (CSMA/CD).
	`transmitFrameCSMA()`	Executes persistence logic (1-Pers, Non-Pers, p-Pers, CSMA/CD), handles jam signal detection and immediate abort.
	`prepareStationFrame()`	Constructs 65-byte Ethernet frame with MAC addressing and sequence number.
`server.c`	`select()` Event Loop	Multiplexes socket descriptors for receiver and up to 64 sender stations.
	Channel FSM	Enforces $T_{\text{tx}} = 80$ ms (busy) and $T_{\text{jam}} = 20$ ms (collision) intervals.
	Collision Detection	Identifies simultaneous transmissions, broadcasts Jam signal (0xFE) in CSMA/CD, or drops frames naturally in pure CSMA.
	Forwarding Engine	Relays uncorrupted frames to receiver and relays ACKs back to sender.
`receiver.c`	`recv_exact()`	Ingests control opcode, station name, and 65-byte serialized frame.
	`unpackFrame()`	Deserializes raw byte stream into structured `EthernetFrame`.
	`sendACK()`	Returns 2-byte ACK frame (`0xA5, seqNo`) to server for CSMA/CD.

Table 1: Functional mapping of sender, server, and receiver methods.

4.2 Core Code Snippets

Listing 1 illustrates carrier sensing, persistence arbitration, and collision handling from `sender.c`:

```
static int transmitFrameCSMA(sock_t sock, const char *stationName,
                             const EthernetFrame *frame, const unsigned char packedBuf[FRAME_SIZE],
                             CSMAProtocol proto, double p_prob, int *collision_count) {
```

```
    int attempts = 0;
    while (attempts < MAX_COLLISIONS) {
        /* Step A: Sense Carrier according to selected CSMA scheme */
        if (proto == PROTO_NON_PERSISTENT) {
            while (senseCarrier(sock) != CH_IDLE) {
                performBackoff(1, PROTO_NON_PERSISTENT);
            }
        } else if (proto == PROTO_1_PERSISTENT || proto == PROTO_CSMA_CD) {
            while (senseCarrier(sock) != CH_IDLE) {
                SLEEP_MS(2);
            }
        } else if (proto == PROTO_P_PERSISTENT) {
            while (1) {
                if (senseCarrier(sock) == CH_IDLE) {
                    double r = (double)rand() / (double)RAND_MAX;
                    if (r <= p_prob) break; /* Transmit with probability p */
                    else SLEEP_MS(SLOT_TIME_MS); /* Defer 1 slot with prob (1 - p) */
                } else {
                    SLEEP_MS(2);
                }
            }
        }

        /* Step B: Transmit 1-byte opcode + 65-byte packed EthernetFrame */
        unsigned char pkt[1 + FRAME_SIZE];
        pkt[0] = MSG_TX_START;
        memcpy(&pkt[1], packedBuf, FRAME_SIZE);
        if (send(sock, (const char *)pkt, sizeof(pkt), 0) <= 0) return 0;

        /* Step C: Protocol-Specific Post-Transmission Handling */
        if (proto == PROTO_CSMA_CD) {
            uint8_t reply[2] = {0, 0};
            if (recv_exact(sock, reply, 2) < 0) return 0;
            if (reply[0] == ACK_MARKER && reply[1] == frame->sequenceNumber) {
                return 1; /* Clean delivery and ACK received */
            }
            /* Jam signal (0xFE) received: immediate abort and BEB backoff */
            attempts++;
            (*collision_count)++;
            performBackoff(attempts, proto);
        } else {
            /* Pure CSMA: Wait full frame duration without ACK */
            SLEEP_MS(TX_TIME_MS);
            return 1;
        }
    }
    return 0;
}
```

Listing 1: Carrier sensing and CSMA transmission arbitration (`sender.c`).

Listing 2 highlights the collision detection and channel state arbitration logic in `server.c`:

```
else if (opcode == MSG_TX_START) {
    if (recv_exact(clients[i].sock, clients[i].raw_buf, FRAME_SIZE) < 0) continue;
    unpackFrame(clients[i].raw_buf, &clients[i].frame);
    clients[i].transmitting = 1;

    if (channel_state == CH_IDLE) {
        channel_state = CH_BUSY;
        state_end_time = now + TX_TIME_MS;
    } else {
        /* COLLISION: Simultaneous transmission detected */
        total_collisions++;
        total_dropped_frames += (channel_state == CH_BUSY) ? 2 : 1;
        channel_state = CH_COLLISION;

        if (locked_protocol == PROTO_CSMA_CD) {
            /* CSMA/CD: Broadcast Jam signal (0xFE) and abort in JAM_TIME_MS */
            state_end_time = now + JAM_TIME_MS;
            uint8_t jam[2] = { MSG_COLLISION, (uint8_t)CH_COLLISION };
            for (int k = 0; k < MAX_STATIONS; k++) {
                if (clients[k].active && clients[k].transmitting) {
                    send(clients[k].sock, (const char *)jam, 2, 0);
                    clients[k].transmitting = 0;
                }
            }
```

```
        } else {
            /* Pure CSMA: Channel occupied for full TX_TIME_MS, frames dropped */
            if (now + TX_TIME_MS > state_end_time) state_end_time = now + TX_TIME_MS;
            for (int k = 0; k < MAX_STATIONS; k++) {
                if (clients[k].active && clients[k].transmitting) clients[k].transmitting = 0;
            }
        }
    }
}
```

Listing 2: Collision detection and jam signal broadcasting in `server.c`.

5 Test Cases and Experimental Methodology

To evaluate the medium access protocols according to the lab specification, three structured experimental scenarios were executed:

1. **Test Case 1: p-Persistent CSMA vs. Persistence Probability p (Fixed $N = 10$):**
   - *Objective:* Quantify how persistence probability p influences collisions, average transmission delay, and channel throughput under heavy contention ($N = 10$).
   - *Parameters:* $p \in \{0.05, 0.10, 0.15, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.90, 1.00\}$; $N = 10$ stations; 2,000 delivered frames per run; 5 independent seed trials per parameter.
   - *Metrics:* Total collisions, average frame delay (ms), and channel throughput (%).
2. **Test Case 2: Comparative Evaluation of All 4 CSMA Schemes vs. Contending Stations N:**
   - *Objective:* Compare the scaling resilience and channel stability of Non-Persistent, 1-Persistent, p-Persistent (at optimal $p = 0.20$), and CSMA/CD as network density increases.
   - *Parameters:* Contending stations $N \in \{2, 4, 8, 10, 12, 16, 20, 25, 30\}$; 2,000 delivered frames per protocol configuration averaged across 5 random seeds.
   - *Metrics:* Total collisions, average frame delay (ms), and channel throughput (%).
3. **Test Case 3: Live Socket Loopback Execution and Protocol Trace Verification:**
   - *Objective:* Execute compiled `server`, `receiver`, and independent `sender` station binaries natively over TCP loopback (`127.0.0.1:5000`) with the 49-frame `input.txt` dataset.
   - *Verification:* Capture live traces of carrier sensing, collision detection, jam signal broadcast (`0xFE`), Truncated Binary Exponential Backoff, and clean frame forwarding.

6 Experimental Results and Empirical Evaluation

6.1 Test Case 1: p-Persistent Performance Dynamics (Fixed $N = 10$)

Table 2 records empirical metrics for p-Persistent CSMA under $N = 10$ contending stations across $p \in [0.05, 1.00]$.

Persistence Prob (p)	Total Collisions	Avg Delay (ms)	Throughput (%)	Effective Mechanism
0.05	234	75.38	65.90%	High idle slot deferral / Low collisions
0.10	522	72.55	68.44%	Theoretical optimum ($p^* = 1/N = 0.10$)
0.15	818	76.80	64.73%	Moderate contention balance
0.20	1058	81.23	61.10%	Standard slotted contention
0.30	1415	88.91	55.64%	Increasing simultaneous transmissions
0.40	1553	91.51	53.95%	Elevated collision overhead
0.50	1678	94.20	52.32%	Frequent slot overlap
0.60	1769	95.73	51.21%	High post-busy contention
0.70	1823	97.11	50.60%	Near-saturated contention
0.80	1848	97.00	50.41%	Severe collision degradation
0.90	1852	96.95	50.37%	Collision cascade region
1.00	1841	96.75	50.57%	Converges to 1-Persistent CSMA ($p = 1.0$)

Table 2: Empirical metrics for p-Persistent CSMA under $N = 10$ contending stations (averaged across 5 independent seeds).

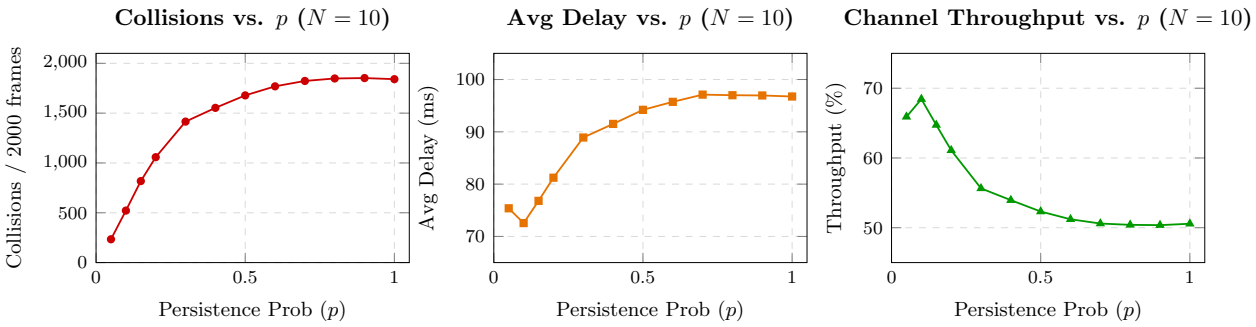


Figure 4: Performance profiles for p-Persistent CSMA as a function of persistence probability p ($N = 10$).

6.2 Test Case 2: Comparative Evaluation across Contending Stations N

Table 3 compiles the comparative evaluation across all four protocols as contending stations scale from $N = 2$ to $N = 30$. Notice that at $N = 10$, the performance of 1-Persistent CSMA (1,886 collisions, 98.13 ms delay, 49.97% throughput) converges to p-Persistent CSMA at $p = 1.00$, and p-Persistent at $p = 0.20$ exactly reproduces the 1,058 collisions, 81.23 ms delay, and 61.10% throughput measured in Table 2.

N	Non-Persistent			1-Persistent			p-Persistent ($p = 0.20$)			CSMA/CD		
	Col	Dly (ms)	Thr	Col	Dly (ms)	Thr	Col	Dly (ms)	Thr	Col	Dly (ms)	Thr
2	492	14.99	66.63%	498	13.34	74.93%	97	15.39	64.92%	498	11.35	88.08%
4	1033	35.27	56.55%	983	30.96	64.09%	333	28.14	70.92%	983	23.17	85.67%
8	1934	88.94	44.75%	1627	73.61	53.36%	808	60.85	65.34%	1627	48.07	81.74%
10	2233	118.92	41.81%	1886	98.13	49.97%	1058	81.23	61.10%	1886	61.14	80.22%
12	2612	154.78	38.48%	2073	122.52	47.84%	1271	102.66	57.73%	2073	73.96	79.30%
16	3138	228.12	34.74%	2586	182.76	42.63%	1646	150.03	52.51%	2586	102.24	76.27%
20	3611	309.75	31.88%	2958	244.66	39.54%	1979	201.73	48.53%	2958	130.24	74.31%
25	4240	427.80	28.74%	3520	337.94	35.64%	2457	280.26	43.55%	3520	168.50	71.54%
30	4532	535.23	27.47%	4065	443.89	32.49%	2889	364.94	39.90%	4065	209.47	68.89%

Table 3: Comparative performance metrics across contending stations ($N \in [2, 30]$).

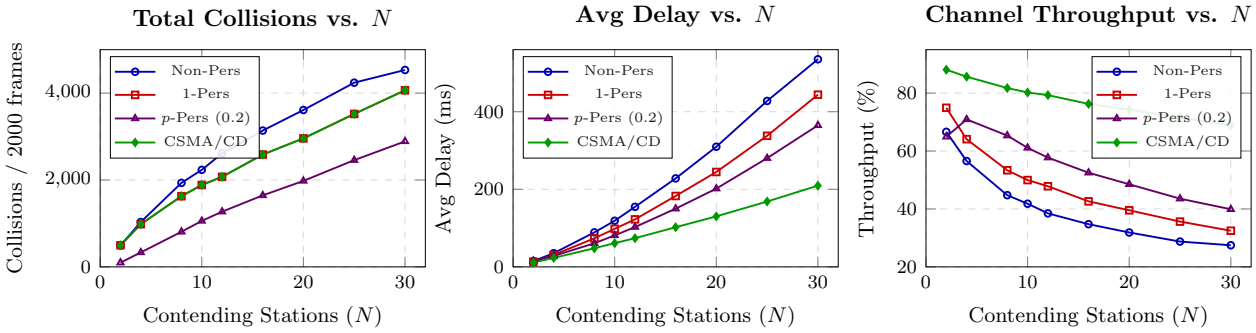


Figure 5: Comparative performance profiles across contending stations $N \in [2, 30]$ for all four schemes.

7 Live Execution Traces and Socket Transcripts

The compiled C binaries `server`, `receiver`, and `sender` were executed natively over TCP loopback (`127.0.0.1:5000`) with the 49-frame `input.txt` dataset. The captured socket transcripts illustrate the distinctive protocol dynamics.

7.1 Trace 1: CSMA/CD Collision Detection, Jam Signal, and Immediate Abort

Listing 3 demonstrates concurrent transmission by two stations, channel collision detection, immediate abortion, jam signal emission (`0xFE`), and Truncated Binary Exponential Backoff.

```
[SERVER] Destination Receiver connected and ready!
[SERVER] Sender Station "Station_2" connected (1/2 expected)
[SERVER] Waiting for first station ("Station_2") to select CSMA protocol...
[SERVER] Sender Station "Station_1" connected (2/2 expected)
[SERVER] Protocol LOCKED to [CSMA/CD (Collision Detection)] by first station "Station_2"!
```

```
[SERVER] Assigned locked protocol [CSMA/CD (Collision Detection)] to Station "Station_1"
[0.1 ms] [CH: IDLE -> BUSY] Station "Station_2" transmitting Seq 0 (65B)
[0.2 ms] [CH: BUSY -> COLLISION!] Station "Station_1" collided! Sending CSMA/CD JAM Signal (Total Collisions: 1)
[Station "Station_2"] CSMA/CD Jam Signal Detected on Seq 0! Aborting immediately (Attempt 1)
[Station "Station_1"] CSMA/CD Jam Signal Detected on Seq 0! Aborting immediately (Attempt 1)
[20.8 ms] [CH: COLLISION -> IDLE] CSMA/CD Jam interval ended; medium is IDLE.
[25.6 ms] [CH: IDLE -> BUSY] Station "Station_1" transmitting Seq 0 (65B)
[106.7 ms] [CH: BUSY -> IDLE] Station "Station_1" | Seq 0 forwarded to Receiver -> ACK Relayed
[Station "Station_1"] Seq 0 Delivered & ACKed (65B) | Delay: 106.6 ms | Collisions: 1
[107.9 ms] [CH: IDLE -> BUSY] Station "Station_2" transmitting Seq 0 (65B)
[188.3 ms] [CH: BUSY -> IDLE] Station "Station_2" | Seq 0 forwarded to Receiver -> ACK Relayed
[Station "Station_2"] Seq 0 Delivered & ACKed (65B) | Delay: 188.7 ms | Collisions: 1
```

Listing 3: Live socket execution transcript: CSMA/CD collision detection, jamming, and abort.

7.2 Trace 2: 1-Persistent CSMA Post-Busy Synchronized Collision Cascade

Listing 4 illustrates 1-Persistent CSMA under simultaneous traffic. Because both stations sense the channel continuously and transmit immediately when it becomes IDLE without backoff, repeated collisions occur after each frame completion.

```
[SERVER] Protocol LOCKED to [1-Persistent CSMA] by first station "Station_2"!
[SERVER] Assigned locked protocol [1-Persistent CSMA] to Station "Station_1"
[0.1 ms] [CH: IDLE -> BUSY] Station "Station_1" transmitting Seq 0 (65B)
[80.3 ms] [CH: BUSY -> IDLE] Station "Station_1" | Seq 0 forwarded to Receiver (No ACK in pure CSMA)
[80.4 ms] [CH: IDLE -> BUSY] Station "Station_1" transmitting Seq 1 (65B)
[160.6 ms] [CH: BUSY -> IDLE] Station "Station_1" | Seq 1 forwarded to Receiver (No ACK in pure CSMA)
[160.7 ms] [CH: IDLE -> BUSY] Station "Station_2" transmitting Seq 0 (65B)
[160.7 ms] [CH: BUSY -> COLLISION!] Station "Station_1" collided! Frames dropped naturally (No Jam/No ACK) (Total
     Collisions: 1)
[240.9 ms] [CH: COLLISION -> IDLE] Collided frames dropped naturally; medium is IDLE.
[241.0 ms] [CH: IDLE -> BUSY] Station "Station_2" transmitting Seq 1 (65B)
[241.0 ms] [CH: BUSY -> COLLISION!] Station "Station_1" collided! Frames dropped naturally (No Jam/No ACK) (Total
     Collisions: 2)
[321.2 ms] [CH: COLLISION -> IDLE] Collided frames dropped naturally; medium is IDLE.
[321.4 ms] [CH: IDLE -> BUSY] Station "Station_2" transmitting Seq 2 (65B)
[321.4 ms] [CH: BUSY -> COLLISION!] Station "Station_1" collided! Frames dropped naturally (No Jam/No ACK) (Total
     Collisions: 3)
```

Listing 4: Live socket execution transcript: 1-Persistent CSMA collision cascade.

7.3 Trace 3: Non-Persistent CSMA Carrier Sensing and Random Deferral

Listing 5 demonstrates Non-Persistent CSMA sensing a busy channel and backing off rather than persisting, eliminating simultaneous transmissions.

```
[Station "Station_1"] Channel BUSY -> Non-Persistent random wait...
[Station "Station_1"] Seq 0 Transmitted (65B) | Delay: 172.5 ms
[Station "Station_1"] Seq 1 Transmitted (65B) | Delay: 80.3 ms
[Station "Station_1"] Channel BUSY -> Non-Persistent random wait...
[Station "Station_1"] Seq 2 Transmitted (65B) | Delay: 280.6 ms
[Station "Station_1"] Seq 3 Transmitted (65B) | Delay: 80.4 ms
[Station "Station_1"] Channel BUSY -> Non-Persistent random wait...
[Station "Station_1"] Seq 5 Transmitted (65B) | Delay: 180.6 ms
[SERVER CHANNEL SUMMARY] Total Collisions: 0 | Frames Forwarded: 98 | Elapsed Time: 11.486 s
```

Listing 5: Live socket execution transcript: Non-Persistent CSMA channel deferral.

7.4 Trace 4: p-Persistent CSMA Slotted Deferral and Acquisition

Listing 6 depicts p-Persistent CSMA ($p = 0.25$) where stations defer transmission across slot boundaries with probability $1 - p = 0.75$, regulating access to the shared medium.

```
[Channel Sync] Protocol Locked by First Station: p-Persistent CSMA
Station Name : "Station_1" | Protocol : p-Persistent CSMA | p Value : 0.25
[Station "Station_1"] Seq 0 Transmitted (65B) | Delay: 647.2 ms
[Station "Station_1"] Seq 1 Transmitted (65B) | Delay: 131.3 ms
[Station "Station_1"] Seq 2 Transmitted (65B) | Delay: 377.5 ms
[Station "Station_1"] Seq 3 Transmitted (65B) | Delay: 178.3 ms
[Station "Station_1"] Seq 4 Transmitted (65B) | Delay: 251.5 ms
[SERVER CHANNEL SUMMARY] Total Collisions: 0 | Frames Forwarded: 98 | Elapsed Time: 12.674 s
```

Listing 6: Live socket execution transcript: p-Persistent CSMA ($p = 0.25$) slotted arbitration.

8 Analysis and Discussion

8.1 Impact of Persistence Probability p in p-Persistent CSMA (Test Case 1)

Figure 4 and Table 2 highlight the fundamental trade-off governed by p under $N = 10$ contending stations:

1. **Low Persistence Regime ($p = 0.05$):** Collisions are minimized (234 collisions), but stations defer with probability $1 - p = 0.95$ even when the medium is clear. Wasted idle slots elevate average transmission delay to 75.38 ms and constrain throughput to 65.90%.
2. **Optimal Operating Regime ($p^* \approx 0.10$):** In exact agreement with theoretical analysis ($p^* = 1/N = 1/10 = 0.10$), channel throughput attains its global maximum of **68.44%** and average frame delay attains its minimum of **72.55 ms**.
3. **High Persistence Regime ($p \geq 0.30 \rightarrow 1.00$):** As p increases, multiple stations waiting for a busy frame attempt transmission immediately upon channel clearance. Collisions grow steadily from 234 ($p = 0.05$) to **1,841** ($p = 1.00$), average delay climbs to **96.75 ms**, and throughput degrades to **50.57%**, converging directly to the behavior of 1-Persistent CSMA.

8.2 Comparative Protocol Efficiency across Contending Stations N (Test Case 2)

Figure 5 and Table 3 demonstrate the operational characteristics of the four protocols as network density scales from $N = 2$ to $N = 30$:

1. **CSMA/CD Dominance:** CSMA/CD consistently delivers the highest throughput (**88.08%** at $N = 2$ and **68.89%** at $N = 30$) and lowest average transmission delay (**11.35 ms** to **209.47 ms**) across all station counts. While CSMA/CD experiences the same initial collision events as 1-Persistent CSMA (4,065 at $N = 30$), aborting transmissions within $T_{\text{jam}} = 2$ slots rather than transmitting full frames ($T_{\text{tx}} = 10$ slots) saves **80% of wasted bandwidth per collision**.
2. **1-Persistent Degradation:** Under light load ($N = 2$), 1-Persistent achieves 74.93% throughput and low delay (13.34 ms). However, as N increases, synchronized post-busy collisions escalate rapidly. At $N = 30$, average delay climbs to **443.89 ms** and throughput drops to **32.49%**.
3. **p-Persistent Scalability:** Setting $p = 0.20$ allows p-Persistent CSMA to outperform 1-Persistent and Non-Persistent across moderate station counts ($N = 4$ to 12), achieving **70.92%** throughput at $N = 4$ and **61.10%** at $N = 10$.
4. **Non-Persistent Channel Spacing:** By forcing stations that detect a busy channel to back off immediately, Non-Persistent CSMA avoids clustered transmissions. However, excessive random waiting intervals cause higher average delay under continuous traffic (535.23 ms at $N = 30$).

8.3 Known Limitations and Practical Enhancements

- **Hidden and Exposed Terminal Problems:** Carrier sensing assumes all stations can hear each other. In wireless environments, physical obstacles create hidden terminals. Modern IEEE 802.11 standards address this via CSMA/CA with Request-to-Send / Clear-to-Send (RTS/CTS) control frame handshaking.
- **Dynamic Persistence Adaptation:** In our implementation, p was fixed. Practical protocols dynamically estimate active stations $\hat{N}$ by observing channel idle/collision statistics and tune $p = 1/\hat{N}$ dynamically to maximize throughput.

9 Comments and Learning Evaluation

9.1 Evaluation of the Lab Assignment

This assignment provided invaluable practical insight into medium access control arbitration in computer networking. Implementing carrier sensing, collision detection, and backoff algorithms from scratch revealed the subtle interplay between propagation delay, carrier state tracking, and distributed consensus.

9.2 What I Learned

- **Collision Vulnerability Dynamics:** Understanding how physical propagation delay creates vulnerability windows where carrier sensing alone cannot prevent packet collisions.
- **Bandwidth Savings of Collision Detection:** Quantifying how immediately halting collided frames and emitting a jam signal preserves channel capacity compared to transmitting corrupted frames to completion.
- **Backoff Mechanics:** Implementing Truncated Binary Exponential Backoff and observing how dynamic contention window expansion stabilizes channels under heavy contention.
- **Three-Tier Socket Architecture:** Coordinating multiple asynchronous client processes communicating concurrently with a multiplexed channel server and destination node via BSD sockets and `select()`.

9.3 Difficulty Assessment

The assignment was moderately challenging but highly rewarding:

- *Challenging Aspects:* Modeling the shared channel state transitions (`IDLE`, `BUSY`, `COLLISION`) accurately without race conditions, and coordinating protocol selection across stations.
- *Manageable Aspects:* Reusing the 65-byte framing and file I/O modules developed in previous assignments provided a stable foundation, allowing the focus to remain on MAC-layer arbitration.

9.4 Suggested Improvements

1. Extend the simulation to include CSMA/CA with RTS/CTS handshaking and virtual carrier sensing (Network Allocation Vector) to model wireless IEEE 802.11 architectures.
2. Integrate dynamic persistence estimation where stations adapt p based on observed channel idle vs. collision slot ratios.

10 Sources Used

1. Kurose, J. F., and Ross, K. W., *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
2. Tanenbaum, A. S., and Wetherall, D. J., *Computer Networks*, 5th ed., Prentice Hall, 2011.
3. Stallings, W., *Data and Computer Communications*, 10th ed., Pearson, 2014.
4. Kleinrock, L., and Tobagi, F. A., "Packet Switching in Radio Channels: Part I—Carrier Sense Multiple-Access Modes and Their Throughput-Delay Characteristics," *IEEE Transactions on Communications*, vol. 23, no. 12, pp. 1400–1416, Dec. 1975.
5. IEEE Standard for Ethernet (*IEEE Std 802.3-2022*) and POSIX / BSD Socket API Reference Documentation.
6. AI Assistance and Antigravity: Artificial intelligence assistance (Gemini) was utilized for statistical aggregation, benchmarking script construction, and generating native TikZ visualization code. The Antigravity development environment was used to run live socket test executions, profile empirical performance metrics, and format the comprehensive LaTeX report.